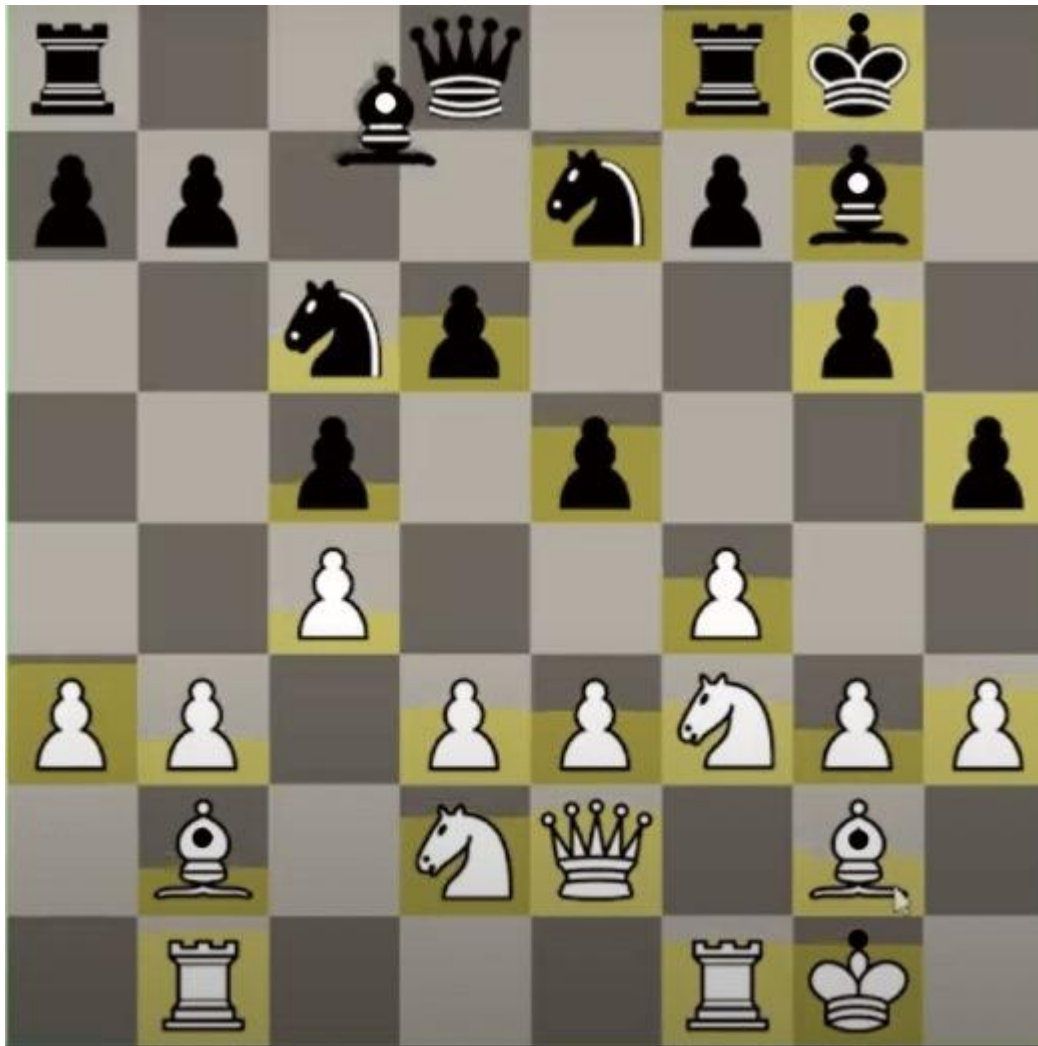


CTD 25

KongFu Chess Server

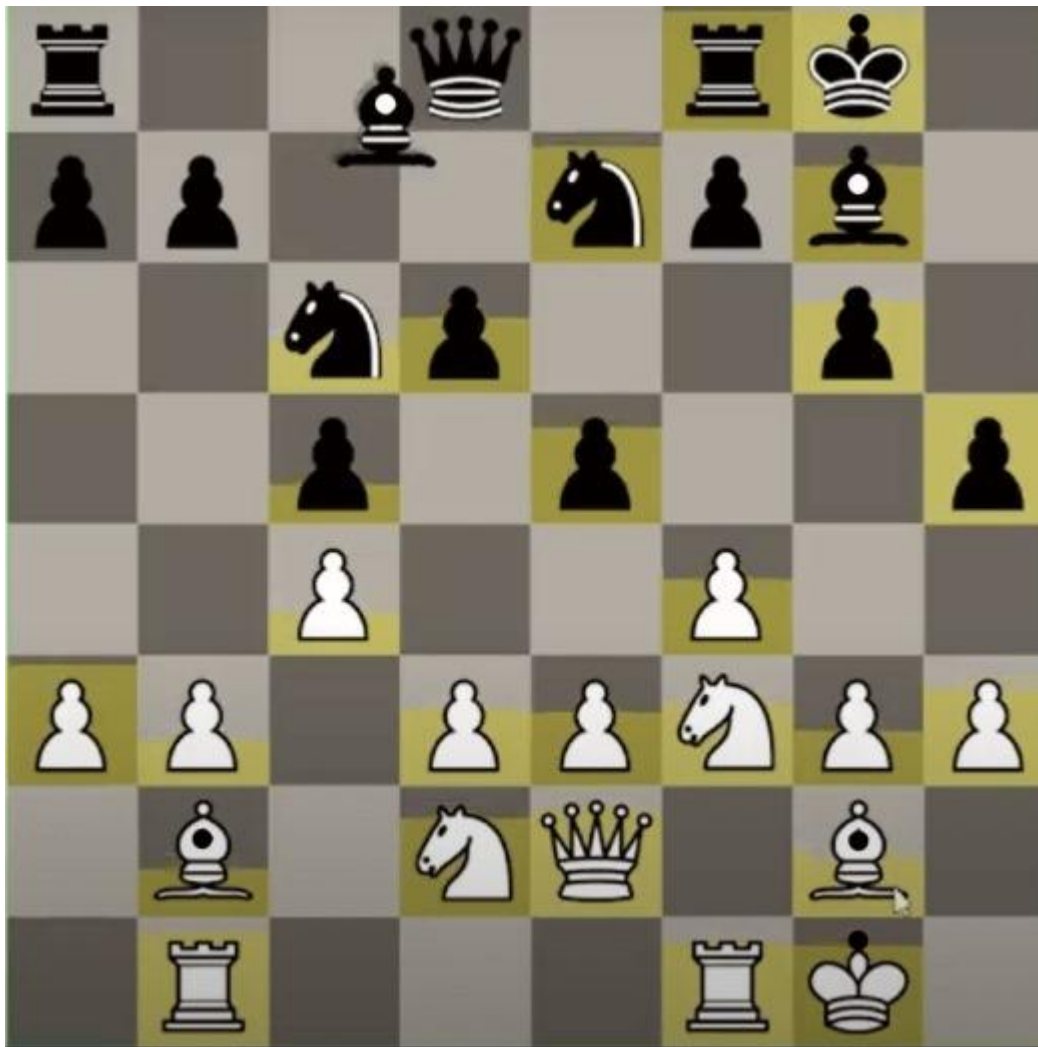
Game Overview

1. Simultaneous play
2. After each move a piece has to “rest” before it can move again.
3. Win = Capturing the King. No such thing as Check or Mate.



Game Extras we'd add.

1. Another command:
Jump + short break.
2. Allow additional piece types
E.g. Quadcopter (slower than others, but can move to any square in $[-2, +2]$)
3. Customizable animations



Additional Requirements

1. Moves log
2. Presenting score
= “cost” of captured pieces.
3. Presenting player names

Black

Time	Move
00:04.105	e5
00:09.642	Nc6
00:15.998	a6
00:20.781	Nf6
00:26.203	Be7
00:31.744	d6
00:37.215	O-O
00:43.668	b5
00:50.013	Qe8
00:57.419	Rd8
01:04.756	exd4
01:11.279	Qh5
01:18.662	Nxe4
01:25.030	f5
01:32.412	gxf6

Name: Chicko Miko

Score: 0



Score: 0

Name: Musti Shusti

White

Time	Move
00:02.314	e4
00:06.927	Nf3
00:13.401	Bb5+
00:18.222	O-O
00:23.589	Re1
00:28.044	d4
00:33.617	Nc3
00:40.102	Bxc6
00:46.455	Qe2
00:53.880	Bg5
01:01.132	Rad1
01:08.577	dxe5
01:15.091	Ne4
01:22.448	Qf3
01:29.733	Nxf6+

Non Functional Requirements

Potentially,
scalable to millions of players
simultaneously.

Black

Time	Move
00:04.105	e5
00:09.642	Nc6
00:15.998	a6
00:20.781	Nf6
00:26.203	Be7
00:31.744	d6
00:37.215	O-O
00:43.668	b5
00:50.013	Qe8
00:57.419	Rd8
01:04.756	exd4
01:11.279	Qh5
01:18.662	Nxe4
01:25.030	f5
01:32.412	gxf6

Name: Chicko Miko

Score: 0



Score: 0

Name: Musti Shusti

White

Time	Move
00:02.314	e4
00:06.927	Nf3
00:13.401	Bb5+
00:18.222	O-O
00:23.589	Re1
00:28.044	d4
00:33.617	Nc3
00:40.102	Bxc6
00:46.455	Qe2
00:53.880	Bg5
01:01.132	Rad1
01:08.577	dxe5
01:15.091	Ne4
01:22.448	Qf3
01:29.733	Nxf6+

Development Process

1. Total ~10 work days.
2. Development is done in iterations.
3. Submission is done as VPL answers (multiple files).
4. Required: Code + Test coverage
5. After each iteration, a solution will be given to the students
(Reviewers get it beforehand)
6. Students can use LLMs as much as they want. Yet they should be responsible for their code.

Iteration A: Core Logic. 3 Days

1. **Given:** Image reading and presentation library (per lang, same interface):

```
class Img:
    def read(path)    # with transparency
    def draw_on(other_img, x, y) # with transparency
    def put_text(txt: str, x, y, font_size) # write text on myself
    def show() #presents the image in game window
```

2. **Given:** Moves + Animation images in a predefined folder structure



Iteration A: Core Logic. 3 Days

Given specific OOD instructions (design patterns and libraries),
implement a game loop:

```
def run():
```

```
    self.start_user_input_thread() # listens to keyboard and fills user_input_queue  
    self.announce_game_start(gamestart_time) # game start graphics
```

```
    while not self.is_win():
```

```
        while self.user_input_queue not empty:
```

```
            command = self.user_input_queue.pop() # user input in "world" coords, not screen coords  
            self.process_input(command) # serialized command. e.g. WQMe2e5 = White Queen Move e2 -> e5
```

```
        self.draw(); # show current game position in GUI
```

```
        self.resolve_collisions()
```

```
        self.check_win()
```

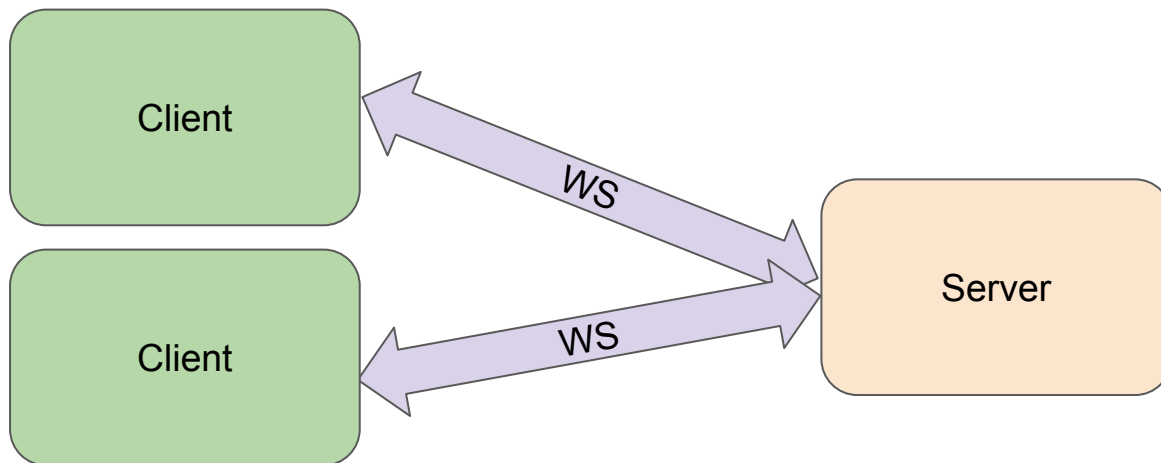
```
    self.announce_win(); # game end graphics
```


Iteration B: BUS. 2 Days

1. Implement pub/sub bus.
2. Use the bus for:
 - a. Update Scores
 - b. Update Move Logs
 - c. Adding sound
 - d. Game start/end animations

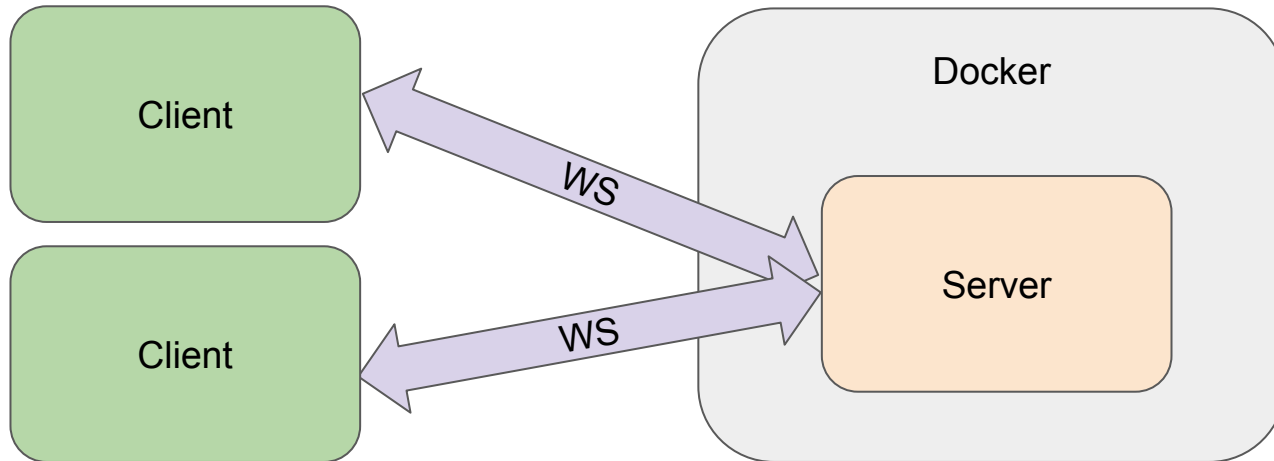
Iteration C: Single-process server. 1 Days

1. Simple local server, WebSocket communication.
2. Sending commands (WQe2e5); Receiving game state.



Iteration D: Dockerization . 1 Day

1. Dockerization of the single process server.



Iteration E: Game Server. 3 Days

